EECS 182 Deep Neural Networks
Spring 2025 Anant Sahai

Final Exam (Spring 2025)

Exam location: VLSB

PRINT your student ID: _____

PRINT AND SIGN your name: _____, _____ _____
(last) (first) (signature)

PRINT your discussion section: _____
Row Number (front row is 1): _____ Seat Number (left most is 1): _____

Name and SID of the person to your left: _____

Name and SID of the person to your right: _____

Name and SID of the person in front of you: _____

Name and SID of the person behind you: _____

Section 0: Pre-exam questions (5 points)

1. Honor Code: Please copy the following statement in the space provided below and sign your name below. (1 point or $-\infty$)

As a member of the UC Berkeley community, I act with honesty, integrity, and respect for others. I will follow the rules and do this exam on my own.

2. What's your favorite deep learning topic? (4 points)

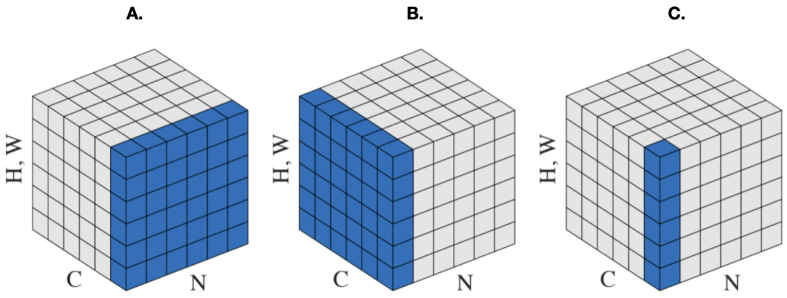
Do not turn this page until the proctor tells you to do so. You can work on Section 0 above before time starts.

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

3. Deja Vu: Normalization in CNNs (4 pts)

Consider the following digram where the shaded blocks are the entries participating in one normalization step for a CNN-type architecture. N represents the mini-batch, H, W represent the different pixels of the "image" at this layer, and C represents different channels.



- **Which one denotes the process of batch normalization?** Please use ■ for your selections.

☐ A ☐ B ☐ C

- **Which one denotes layer normalization?** Please use ■ for your selections.

☐ A ☐ B ☐ C

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

4. Deja Vu: Convolutional Neural Networks (12 pts)

(a) (6 pts) Consider a convolutional neural network layer with:

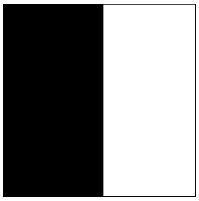
- Input shape: [10, 3, 32, 32] (batch size, number of input channels, input height, input width)
- Number of filters: 64

For each of the following configurations of kernel size, stride, and padding, **calculate the output dimensions** $[n, c, h, w]$ where n is the batch size, c is the number of channels, and h, w are the output height and width. Assume that each layer has the same input shape and number of filters as specified above.

i. Kernel size: 3x3, stride: 1, padding: 1

ii. Kernel size: 4x4, stride: 2, padding: 0

(b) (3 pts) **Design a 3x3 filter that detects vertical edges like the one shown in the image below.**



$$\begin{bmatrix} _ & _ & _ \\ _ & _ & _ \\ _ & _ & _ \end{bmatrix}$$

(c) (3 pts) **Design a 3x3 filter to blur an image.**
(Hint: blurring involves averaging a pixel's value with those of its neighbors.)

$$\begin{bmatrix} _ & _ & _ \\ _ & _ & _ \\ _ & _ & _ \end{bmatrix}$$

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

5. Application matching (14 pts)

Match each machine learning architecture or component (left column) to the correct application (A–I). In the first answer column, look for the "best match" — these will be exclusive with no repeats. In the second column, we can add others for which you think this is a good idea, possibly as a component. We have filled in some boxes for you to give you an idea of what we are expecting. **In both columns, only fill in completely empty boxes.**

Deep Learning Idea	**Best Match (A–H)**	**Others**
1. Transformer		all use cases
2. Modern state-space models	**G**	H, B
3. CNN		A, F
4. U-Net		C
5. Diffusion Models		H
6. GNN (Graph Neural Network)		F, H
7. VAE (Variational Autoencoder)	**F**	
8. RNN (Classic Type)		none

A. Biomedical image segmentation (e.g., tumor boundary detection in MRIs)
B. Simple next-character modeling, with small vocabularies
C. High-fidelity image generation
D. Node classification in research paper citation graphs
E. Image classification (e.g., CIFAR-10, ImageNet)
F. Latent space anomaly detection (e.g., defect detection in manufacturing)
G. Time-series forecasting with long-term dependencies (e.g., ECG signal prediction)
H. Large-scale language modeling (e.g., GPT-style text completion)

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

6. Comparing Sequence Model Architectures (15 pts)

In this question, you will compare three major sequence model architectures—Transformers, modern State Space Models (SSMs), and classic Recurrent Neural Networks (RNNs). Let n be the sequence length and d the dimension of the relevant hidden state or corresponding quantity. For each aspect in the table below, fill in the relevant characteristics for each architecture using the section of the page for the relevant letter. Be concise.

Aspect	**Transformers**	**SSMs**	**RNNs**
Training parallelization for long sequences	Excellent (Highly parallelizable during training)	Great (Can parallelize across sequence dimension)	**(a)**
Training PyTorch Building Block	nn.MultiheadAttention	nn.Conv	nn.RNN
Inference Memory	High for context ($O(nd)$ with KV caching)	**(b)**	Low (Only requires current hidden states $O(d)$)
Modeling long range dependencies	**(c)**	Great (Can model long dependencies with structured state space)	Weak (Struggles with vanishing/exploding gradients)

(a) (3pts)

(b) (3pts)

(c) (3pts)

(d) (6pts) **Explain briefly what key difference in modern state-space-models relative to classic RNNs permits them to train efficiently on modern hardware.**

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

7. LoRA (18 pts)

A common strategy for adapting a large pre-trained model to a new task is to update only a subset of its parameters, keeping the rest frozen. Low-Rank Adaptation (LoRA) offers a more flexible approach to this idea. In this problem, we focus on a single weight matrix W with m rows and ℓ columns, where W_0 is the pre-trained value. During LoRA-based training, W is replaced by $W_0 + AB$, where W_0 remains frozen and only A and B are learnable. Here, A is an $m \times k$ matrix and B is a $k \times \ell$ matrix, typically with $k \ll \min(m, \ell)$.

(a) (5pts) Suppose you are using LoRA to adapt a pretrained deep neural net to a new task and observe that the model is "underfitting" the training data. **What would you try to adjust in the LoRA to get better performance?**

(b) (3pts) Suppose both A and B are initialized to all zeros. **Why will this cause problems for LoRA-based finetuning?**

Remember, this is going to be trained using SGD-style updates over a training set with a loss function.

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

(c) (5pts) Consider the following pseudocode for LoRA initialization:

```
A = torch.nn.Parameter(torch.empty(m, k))
B = torch.nn.Parameter(torch.empty(k, l))
torch.nn.init.xavier_uniform_(A)
torch.nn.init.xavier_uniform_(B)
```

Why might LoRA fine-tuning not work well with this initialization?

(d) (5pts) **How much memory is required to store the LoRA adaptation weights (**A **and** B**)?** Assume we are using floats (4 bytes per real number) and give your answer in bytes. **How does this compare to storing a single full-rank adaptation matrix?**

 © UCB EECS 182, Spring 2025. All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

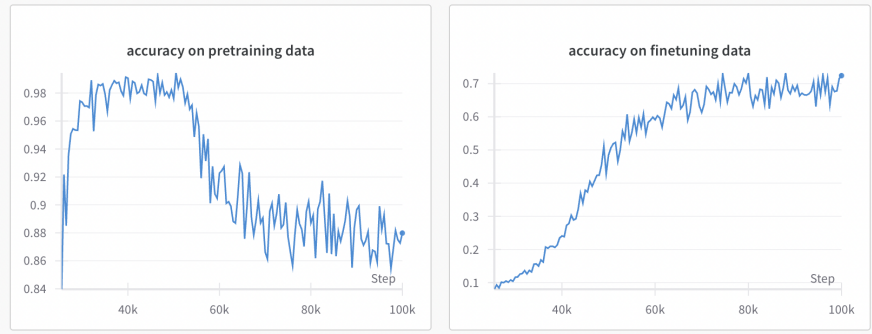
8. Debugging (17 pts)

A botanist on a remote island has found a set of five new, and very different, plant species! Unfortunately, off-the-shelf trained image CNN models (like ones trained on ImageNet) are unable to classify these new plants (since they've just been discovered).

(a) (8pts) **Give two different potential strategies you could use to incorporate the capability to classify these new species into your CNN (assuming that you have access to both the code and data used to originally train it)?** List them in increasing order of training parameter counts.

(b) (3pts) Suppose you choose a co-training strategy: you'll take your pretrained image classifier, add new head parameters for the new species, and jointly fully fine-tune the entire network on both in-distribution pretraining data and your new plant species data.

When you co-train like this, however, you observe the following loss curves:



What is this phenomenon called?

(c) (6pts) **Select all possible approaches you could use to mitigate this phenomenon and have good performance on both tasks:**

- ☐ Add weights to your data, so the model sees a higher percentage of pretraining data in each batch
- ☐ Remove the pretraining data from your fine-tuning process
- ☐ Let it train for longer; as pretraining loss grows, the model may recover
- ☐ Make sure you're using good data augmentations on both data sets.
- ☐ Decrease the learning rate to avoid drastic jumps from the pretrained mode
- ☐ Reduce the amount of training data you're using

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

[Extra page. If you want the work on this page to be graded, make sure you tell us on the problem's main page.]

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

9. Transformer Decoding Optimization (26 pts)

Consider a multi-head attention (MHA) layer in a transformer decoder performing incremental decoding. The attention layer operates on a batch of b sequences, each with a dimension of d. For each of the h heads, the dimensions of query, key, and value projections are $q = \frac{d}{h}$, $k = \frac{d}{h}$, and $v = \frac{d}{h}$ respectively.

The following pseudocode performs the multi-head self-attention operation for a single token generation in the sequence (for each sequence in our batch).

```
def multihead_attention_incremental(x, prev_K, prev_V, W_q, W_k, W_v, W_o):
    """Multi-head Self-Attention (one step).
    Args:
        x: a tensor with shape [b, d] - current token embedding
        prev_K: tensor with shape [b, h, n, k] - cached keys from prev. tokens
        prev_V: tensor with shape [b, h, n, v] - cached values from prev.
            tokens
        W_q: a tensor with shape [h, d, q] - query projection weights
        W_k: a tensor with shape [h, d, k] - key projection weights
        W_v: a tensor with shape [h, d, v] - value projection weights
        W_o: a tensor with shape [h, v, d] - output projection weights
    Returns:
        y: a tensor with shape [b, d] - output embedding
    """
    # Project inputs to query, key, and value
    q = torch.einsum("bd,hdq->bhq", x, W_q)  # [b, h, q]
    k = torch.einsum("bd,hdk->bhk", x, W_k)  # [b, h, k]
    v = torch.einsum("bd,hdv->bhv", x, W_v)  # [b, h, v]

    # Append new key and value to previous cached states
    new_K = torch.cat([prev_K, k.unsqueeze(2)], dim=2)  # [b, h, n+1, q]
    new_V = torch.cat([prev_V, v.unsqueeze(2)], dim=2)  # [b, h, n+1, v]

    # Compute attention scores, apply softmax, and get weighted values
    logits = torch.einsum("bhq,bhnq->bhn", q, new_K)  # [b, h, n+1]
    weights = torch.nn.functional.softmax(logits, dim=-1)  # [b, h, n+1]
    o = torch.einsum("bhn,bhnv->bhv", weights, new_V)  # [b, h, v]

    # Project back to output dimension
    y = torch.einsum("bhv,hvd->bd", o, W_o)  # [b, d]

    return y
```

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

Multi-Query Attention (MQA): Multi-query attention is a variant of multi-head attention where all heads share the same keys and values, reducing memory access and improving efficiency for incremental decoding.

```
def multiquery_attention_incremental(x, prev_K, prev_V, W_q, W_k, W_v, W_o):
    """Multi-query Self-Attention (one step).
    Args:
        x: a tensor with shape [b, d] - current token embedding
        prev_K: tensor with shape [b, n, k] - cached keys (shared across heads)
        prev_V: tensor with shape [b, n, v] - cached values (shared across heads)
        W_q: a tensor with shape _____A____ - query projection weights
        W_k: a tensor with shape _____B____ - key projection weights
        W_v: a tensor with shape [d, v] - value projection weights
        W_o: a tensor with shape [h, v, d] - output projection weights
    Returns:
        y: a tensor with shape [b, d] - output embedding
    """
    # Project input to queries (per head), keys and values (shared)
    q = _____C_____
    k = _____D_____
    v = _____E_____

    # Append new key and value to previous cached states
    new_K = torch.cat([prev_K, k.unsqueeze(1)], dim=1)  # [b, n+1, k]
    new_V = torch.cat([prev_V, v.unsqueeze(1)], dim=1)  # [b, n+1, v]

    # Compute attention scores, apply softmax, and get weighted values
    logits = torch.einsum("bhk,bnk->bhn", q, new_K)     # [b, h, n+1]
    weights = torch.nn.functional.softmax(logits, dim=-1)  # [b, h, n+1]
    o = torch.einsum("bhn,bnv->bhv", weights, new_V)    # [b, h, v]

    # Project back to output dimension
    y = torch.einsum("bhv,hvd->bd", o, W_o)             # [b, d]

    return y
```

First, fill in the blanks in the code/comments above by picking one:

(a) (3pts) **What is A?**

◯ $[h, d, q]$ ◯ $[d, q]$ ◯ $[h, q]$

(b) (3pts) **What is B?**

◯ $[h, d, k]$ ◯ $[d, k]$ ◯ $[h, k]$

(c) (4pts) **What is C?**

◯ `torch.einsum("bd,dq->bhq", x, W_q)`

◯ `torch.einsum("bd,hdq->bhq", x, W_q)`

◯ `torch.einsum("bhd,dq->bhq", x, W_q)`

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

(d) (4pts) **What is D?**

◯ `torch.einsum("bd,dk->bk", x, W_k)`

◯ `torch.einsum("bd,hdk->bhk", x, W_k)`

◯ `torch.einsum("bd,dk->bhk", x, W_k)`

(e) (4pts) **What is E?**

◯ `torch.einsum("bd,hdv->bhv", x, W_v)`

◯ `torch.einsum("bd,dv->bhv", x, W_v)`

◯ `torch.einsum("bd,dv->bv", x, W_v)`

(f) (4pts) Recall that for generating a batch of single tokens in multi-head-attention with a cache of n previous tokens, the operations have computational complexity $O(bd^2 + bnd)$ since it is $O(bd^2)$ to do the projections (query, key, value, output) and $O(bnd)$ from attention (dot products with n cached keys/values for each head). The memory access complexity is $O(d^2 + bd + bnd)$ since it is $O(d^2)$ to read weights, $O(bd)$ to read/write current token embeddings (x, q, k, v, o, y), and $O(bnd)$ to read cached keys/values. The MHA arithmetic intensity is thus $\frac{O(bd^2+bnd)}{O(d^2+bd+bnd)}$.

- Both terms are kept; which one dominates depends on the relative size of n and d.
- If n is small, projections dominate; if n is large, memory access for cached keys/values dominates.

Now consider MQA. For generating a single token with a cache of n previous tokens, **what is the computational complexity (in Big-O notation)?**

(g) (4pts) For MQA generating a single token with a cache of n previous tokens, **what is the memory access complexity (in Big-O notation)?**

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

[Extra page. If you want the work on this page to be graded, make sure you tell us on the problem's main page.]

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

10. Almost Deja Vu: Policy Gradient and Reparameterization (25 pts)

In this question, you are asked to derive two gradient estimators for the following objective function:

$$\mathcal{F}(\theta) = \mathbb{E}_{\mathbf{x}\sim p_\theta}\left[f(\mathbf{x})\right], \tag{1}$$

where $\mathbf{x}$ is a random variable that follows the probability distribution of $p_\theta : \mathcal{X} \mapsto \Delta(\mathcal{X})$ that is parameterized by $\theta \in \mathbb{R}^P$, and $f : \mathcal{X} \mapsto \mathbb{R}$ is a function. Consider the specific case where $\mathbf{x}$ is a k-dimensional multivariate Gaussian random variable that is parameterized by the mean $\mu \in \mathbb{R}^k$ under the distribution:

$$p_\mu(\mathbf{x}) = (2\pi\sigma)^{-n/2} \exp\left(-\|\mathbf{x}-\mu\|_2^2/(2\sigma^2)\right), \tag{2}$$

where $\sigma \in \mathbb{R}$ is a scalar constant.

(a) (10pts) We can rewrite the expression for $\mathcal{F}(\mu)$ as an expectation over a standard $k-$dimensional normal $\mathcal{N}(0, \mathbf{I})$ as

$$\mathcal{F}(\mu) = \mathbb{E}_{\mathbf{z}\sim\mathcal{N}(0,\mathbf{I})}\left[f(\sigma\mathbf{z}+\mu)\right] \tag{3}$$

Use this together with an assumption of differentiability of f to **express $\nabla_\mu \mathcal{F}(\mu)$ as an expectation involving the gradient of f and an expectation with a known distribution** and **describe the implications for training a neural network for the mean μ using SGD to minimize the loss (3).**

(Hint: when can a gradient move inside an expectation?)

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

(b) (15pts) Now, you cannot assume the differentiability of f. However, the density (2) is well behaved given μ. **Express $\nabla_\mu \mathcal{F}(\mu)$ as an expectation under $p_\mu(\mathbf{x})$** (i.e., find $g(\mathbf{x})$ in $\nabla \mathbf{F}(\mu) = \mathbb{E}_{\mathbf{x}\sim p_\mu}\left[g(\mathbf{X})\right]$). and **describe the implications for training a neural network for the mean μ using SGD even though the loss f is not differentiable.**

(Hint 1: $\mathbb{E}_{\mathbf{x}\sim p_\theta}\left[f(\mathbf{x})\right] = \int p_\theta(\mathbf{x}) f(\mathbf{x}) d\mathbf{x}$ where the definite integral is over the entire space.)

(Hint 2: For the implications, it might be easier to massage your final answer into a form similar to the previous part.)

 © UCB EECS 182, Spring 2025. All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

11. Roar: SGD in Adam-style directions (35 pts)

In this problem, we're solving an overparameterized least-squares type problem like the one you saw in the homework.

$$X\mathbf{w} = \mathbf{y} \tag{4}$$

where $X = \begin{bmatrix} \mathbf{x}_1^\top \\ \mathbf{x}_2^\top \\ \vdots \\ \mathbf{x}_n^\top \end{bmatrix}$ is a wide matrix with $\mathbf{x}_i$ being d-dimensional vectors and $\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$ is an n-dimensional vector. Here, we assume that X has full row-rank (i.e. the $\mathbf{x}_i$ are linearly independent) and so (4) indeed has solutions. (Note that as $d \gg n$, there are infinitely many solutions.)

As in the homework, we just initialize $\mathbf{w}_0 = \mathbf{0}$.

Recall that the standard SGD update for squared-loss is:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + 2\eta \mathbf{x}_{I_t}(y_{I_t} - \mathbf{x}_{I_t}^\top \mathbf{w}_t) \tag{5}$$

where I_t is a discrete uniform random variable from 1 to n that picks which data point we are using for this step, and η is the learning rate. In the homework, we saw this converged to $\mathbf{w}^*_{SGD} = X^\top (XX^\top)^{-1}\mathbf{y}$.

Instead of doing (5), we move in an Adam-inspired direction $\check{\mathbf{x}}_{I_t}$ in which every entry is either $+1$ or -1 depending on the sign of that position in $\mathbf{x}_{I_t}$. The update we actually apply is:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + 2\eta \check{\mathbf{x}}_{I_t}(y_{I_t} - \mathbf{x}_{I_t}^\top \mathbf{w}_t) \tag{6}$$

For convenience, let the matrix $\check{X}$ be the entrywise signs of X. Assume that $\check{X}$ also has full row rank. Note $\check{\mathbf{x}}_i^\top$ is the i-th row of that matrix $\check{X}$.

(a) (8pts) Assume for a moment that our learning rate η is small enough and that the iterations of (6) converge to a solution $\mathbf{w}^*$ where $X\mathbf{w}^* = \mathbf{y}$. **Give an explicit expression for what $\mathbf{w}^*$ must be given these iterations. Is this the same as where (5) would converge?**

(HINT: What is the subspace in which the iterations of $\mathbf{w}_t$ are moving. What must the updates in (6) be at $\mathbf{w}^*$?)

 © UCB EECS 182, Spring 2025. All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

(b) (4pts) Consider our iterations (6) as compared to those in (5). **Show whether or not we always move at least a little bit in the direction of the actual negative gradient for all given data points.**
(HINT: Look at the inner product between $\mathbf{x}_i$ and $\check{\mathbf{x}}_i$. What does that mean about the direction in which we are moving?)

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

(c) (9pts) To show that our modified SGD (6) will converge with a small enough constant step size η, you decide to choose the stochastic Lyapunov function:

$$\mathcal{L}(\mathbf{w}) = (\mathbf{w} - \mathbf{w}^*)^\top X^\top X(\mathbf{w} - \mathbf{w}^*) = \|X(\mathbf{w} - \mathbf{w}^*)\|^2 \tag{7}$$

When trying to prove convergence via an argument in the style of the homework, you compute

$$\mathcal{L}(\mathbf{w}_{t+1}) = \mathcal{L}(\mathbf{w}_t) + A + B \tag{8}$$

and find that, once you substitute in for (6) and do some algebra, that

$$E[A|\mathbf{w}_t] = -4\eta(\mathbf{w}_t - \mathbf{w}^*)^\top X^\top X E[\check{\mathbf{x}}_I \mathbf{x}_I^\top](\mathbf{w}_t - \mathbf{w}^*) \tag{9}$$

$$E[B|\mathbf{w}_t] = 4\eta^2(\mathbf{w}_t - \mathbf{w}^*)^\top E[\mathbf{x}_I \check{\mathbf{x}}_I^\top X^\top X \check{\mathbf{x}}_I \mathbf{x}_I^\top](\mathbf{w}_t - \mathbf{w}^*) \tag{10}$$

where I is a random variable taking values in the discrete set $\{1, 2, \ldots, n\}$ with probabilities given by $P(I = i) = \frac{1}{n}$.

Show that $E[B|\mathbf{w}_t] \leq c_2\eta^2\mathcal{L}(\mathbf{w}_t)$ for some $c_2 > 0$.

(HINT: What's the norm of $\check{\mathbf{x}}_I$? Recall that the spectral norm of a matrix X is its largest singular value and this bounds how much X can grow the norm of a vector it is acting on. Further recall the outer-product perspective on matrix multiplication so $X^\top X = \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top$.)

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

(d) (9pts) Assume for this part that the $n \times n$ symmetric matrix $\frac{1}{2}(X\check{X}^\top + \check{X}X^\top)$ has all strictly positive real eigenvalues.

Show that $E[A|\mathbf{w}_t] \leq -c_1\eta\mathcal{L}(\mathbf{w}_t)$ **for some** $c_1 > 0$**.**

Together with the last part, this guarantees exponential convergence since $1 - c_1\eta + c_2\eta^2 < 1$ for small enough positive $\eta > 0$.

 © UCB EECS 182, Spring 2025. All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

(e) (5pts) [Note: No dependence on the previous parts] The sign operation can be viewed as a theoretical simplification of the componentwise normalization by average gradient scale operation in Adam. **If there are d parameters, how much memory would be saved in the optimizer if you could eliminate the need to normalize by the average gradient scale? Explain.**

(HINT: How does Adam do component-wise normalization by the average gradient scale? How much memory does that need?)

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

[Extra page. If you want the work on this page to be graded, make sure you tell us on the problem's main page.]

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

12. DDPM/DDIM Fun: From a Gaussian (52 pts)

Consider generative diffusion models under the simplifying assumption that the target distribution we wish to sample from is a Gaussian with zero mean and small variance σ^2, where $\sigma^2 \ll 1$. The forward process is defined to run from time $t = 0$ to $t = 1$.

Suppose the forward diffusion process starts at $X_0 \sim \mathcal{N}(0, \sigma^2)$ and at each small interval of length Δt adds independent Gaussian noise with mean zero and variance $\Delta t \ll \sigma^2$.

(a) (3pts) **What is the distribution of X_1?**
(HINT: Remember, you can think of $\Delta t = \frac{1}{T}$ for some large T and X_1 as the final result after doing T steps of adding independent Gaussian noise to the initial realization of X_0.)

(b) (4pts) Looking at the forward diffusion process around time t, **what are the marginal distributions of $X_{t-\Delta t}$ and X_t?**

(c) (3pts) In the previous part, the conditional distribution of $(X_{t-\Delta t}|X_t = x_t) \sim \mathcal{N}\left(\frac{\sigma^2+t-\Delta t}{\sigma^2+t}x_t, \frac{(\sigma^2+t-\Delta t)\Delta t}{\sigma^2+t}\right)$.
Simplify and approximate the variance $\frac{(\sigma^2+t-\Delta t)\Delta t}{\sigma^2+t}$ of this conditional distribution when $\Delta t \ll \sigma^2 \ll 1$.

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

(d) (8pts) This part asks you to see what happens if you try to do reverse denoising naively without applying stochastic noise at each reverse diffusion step. Suppose we start with a sample $X_1 \sim \mathcal{N}(0,1)$ and iteratively apply only the conditional mean mapping backward to time $t = 0$ to get a sample of $\widehat{X}_0$. **What is the resulting distribution of $\widehat{X}_0$?** Remember that $\Delta t \ll \sigma^2 \ll 1$ if you want to make any approximations or to interpret the result. Show your work.

(HINT 1: Consider $T = \frac{1}{\Delta t}$ and construct the appropriate product to see how the final $\widehat{X}_0$ is distributed. Do you notice any telescoping?)

(HINT 2: You can use the information given in the next part to check your work here.)

 © UCB EECS 182, Spring 2025. All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

(e) (8pts) In the previous part, you should have found that $\widehat{X}_0$ has far too little variance if you don't add independent stochastic noise along the way in DDPM-style reverse diffusion.

Let $T = \frac{1}{\Delta t}$. Suppose now that we add independent $\mathcal{N}(0, \Delta t)$ noise at each of T reverse diffusion steps. It turns out that

$$\text{Var}(\widehat{X}_0) = \left(\frac{\sigma^2}{\sigma^2+1}\right)^2 + \Delta t \sum_{k=0}^{T-1} \left(\frac{\sigma^2}{\sigma^2 + k\Delta t}\right)^2. \tag{11}$$

Take the limit $\Delta t \to 0$ and **write Var$(\widehat{X}_0)$ involving an integral assuming** $0 < \Delta t \ll \sigma^2 \ll 1$**.** And then **evaluate it approximately.**

(HINT 1: For evaluating the integral, $\int \frac{1}{(1+t)^2} dt = C - \frac{1}{1+t}$.)

(HINT 2: You know that the answer should come out to be close to our desired variance of σ^2 and you can use that to check your work.)

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

[Extra page. If you want the work on this page to be graded, make sure you tell us on the problem's main page.]

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

(f) (8pts) Notice that the conditional mean step in DDPM is approximately a $\Delta t/t$ step toward the ideal predictor $\widehat{X}_0^* = \frac{\sigma^2}{\sigma^2+t} X_t$. Namely that

$$\widehat{X}_{t-\Delta t}^{\text{DDPM}} = \widehat{x}_t + \frac{\Delta t}{t}(\widehat{X}_0^* - \widehat{x}_t) + \text{noise} = \left(1 - \frac{\Delta t}{\sigma^2 + t}\right)\widehat{x}_t + \text{noise}$$

This can be incrementally viewed as a deterministic DDPM step of $-\frac{\Delta t}{\sigma^2+t} x_t$ together with a noise step.

DDIM takes no noise step, but a smaller time-varying deterministic step:

$$x_{t-\Delta t}^{\text{DDIM}} = x_t + \eta(t, \Delta t)(\text{deterministic DDPM step}) \quad \text{with } \eta(t, \Delta t) = \frac{\sqrt{t}}{\sqrt{t - \Delta t} + \sqrt{t}}$$

Describe briefly, if you were simply given a dataset of samples $s_1, s_2, \ldots, s_n$ drawn from the same distribution as our desired X_0 distribution, how you would train a neural network to estimate the function $g(x_t, t)$ that in this case, turns out to be $\frac{\sigma^2}{\sigma^2+t} x_t$ in analytic form. **Specifically, what are the inputs to the neural net, how do you generate a batch of them, and how would you compute a loss on the outputs of the neural net** for running an optimization algorithm like AdamW to set the parameters of the neural net?

(g) (6pts) Assuming you had exact analytic form access to $g(x_t, t) = \frac{\sigma^2}{\sigma^2+t} x_t$ in the previous part, **approximate the DDIM step for $\Delta t \ll t$, with no assumption about σ^2 vs t.**

(HINT: Use the fact that $\Delta t \ll t$ to approximate $\eta(t, \Delta t)$ as an appropriate constant.)

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

(h) (12pts) Compute $\widehat{X}_0$ from the random sample $\widehat{X}_1$ (drawn from $\mathcal{N}(0,1)$) by applying all $T = \frac{1}{\Delta t}$ DDIM steps from the previous part and expressing the total contraction as a product. Then turn it into a sum via logarithms and take the $\Delta t \to 0$ limit, using integrals as needed to evaluate the resulting distribution for $\widehat{X}_0$. **Show your work.**

(HINT 1: You know that the answer is supposed to be approximately $\mathcal{N}\left(0, \sigma^2\right)$ under our assumption that $\sigma^2 \ll 1$. Use this to check your work.)

(HINT 2: Remember $\ln(1-x) \approx -x$ if $0 < x \ll 1$.)

(HINT 3: The log of a limit is the limit of logs.)

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

[Extra page. If you want the work on this page to be graded, make sure you tell us on the problem's main page.]

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

[Extra page. If you want the work on this page to be graded, make sure you tell us on the problem's main page.]

 All Rights Reserved. This may not be publicly shared without explicit permission.

PRINT your initials and student ID: _____

[Doodle page! Draw us something if you want or give us suggestions or complaints. You can also use this page to report anything suspicious that you might have noticed.

If needed, you can also use this space to work on problems. But if you want the work on this page to be graded, make sure you tell us on the problem's main page.]

 All Rights Reserved. This may not be publicly shared without explicit permission.